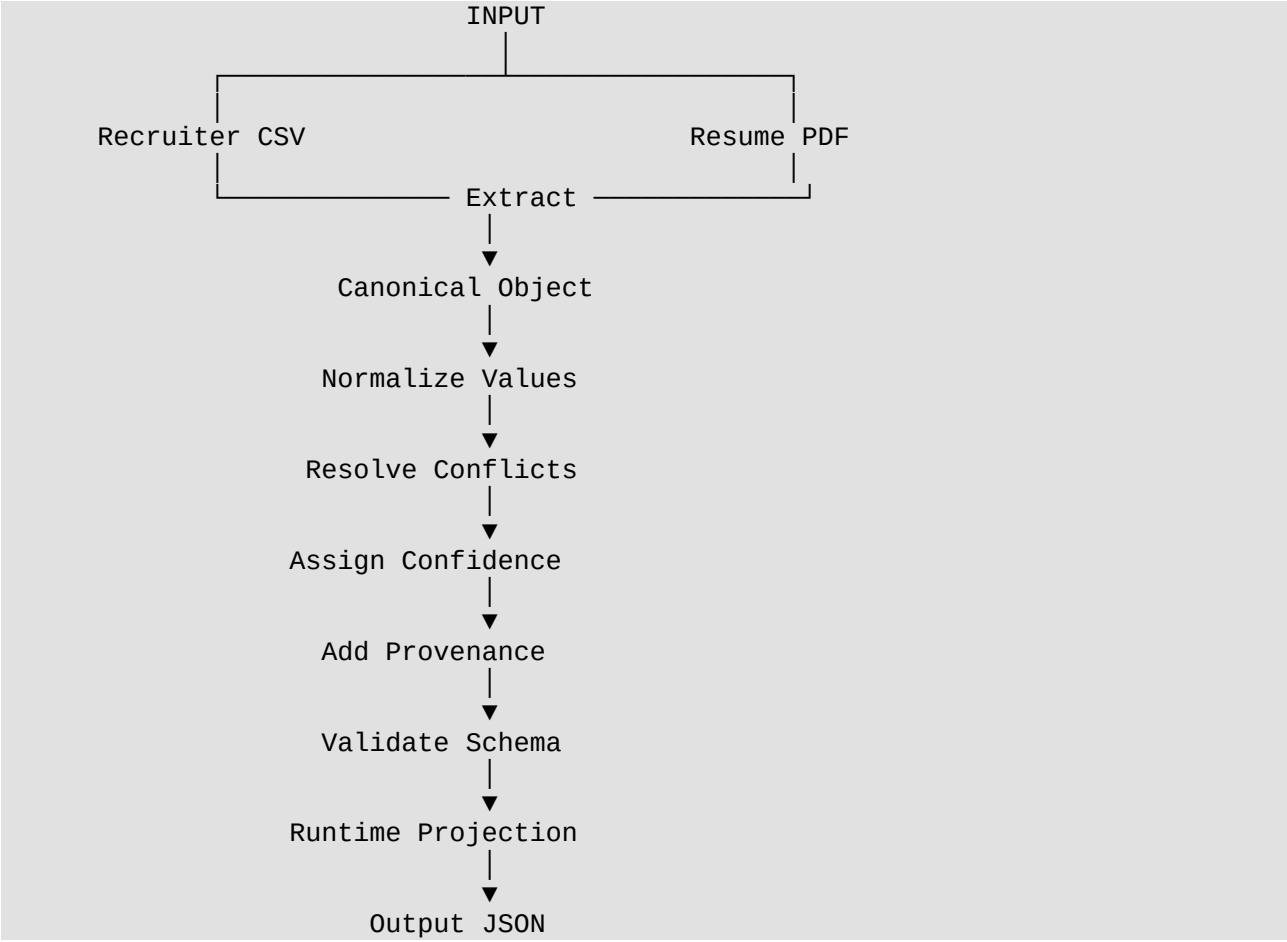
DESIGN DOCUMENT

Eightfold Profile Engine

Author: Ayush Arya

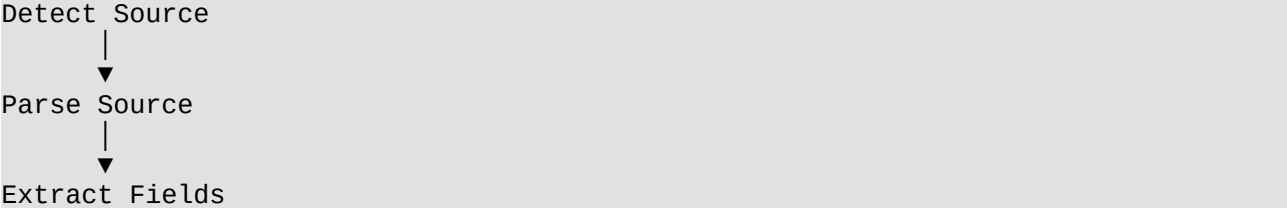
Step 1 – Technical Design

System Pipeline



Internal Processing Pipeline

Every source follows the same processing workflow before contributing to the canonical profile.



```
      │
      ▼
Normalize Values
      │
      ▼
Merge Records
      │
      ▼
Assign Confidence
      │
      ▼
Generate Provenance
      │
      ▼
Schema Validation
      │
      ▼
Runtime Projection
      │
      ▼
Output JSON
```

Canonical Schema

The merge engine stores all candidate information in a single canonical representation.

```
{
    "candidate_id": null,
    "full_name": "string",
    "emails": [],
    "phones": [],
    "location": {
        "city": null,
        "region": null,
        "country": null
    },
    "links": {
        "linkedin": null,
        "github": null,
        "portfolio": null,
        "other": []
    },
    "headline": null,
    "years_experience": null,
    "skills": [],
    "experience": [],
    "education": [],
    "provenance": [],
    "overall_confidence": 0.0
}
```

Step 2 – System Overview

Objective

The Eightfold Profile Engine aggregates candidate information from multiple heterogeneous sources into a unified canonical profile.

The application accepts structured recruiter information and unstructured resume data, extracts useful information, normalizes it, merges conflicting records, validates the final profile, and generates configurable client-specific outputs using runtime projection.

System Components

Module	**Responsibility**
Recruiter Parser	Parse CSV
Resume Parser	Parse PDF
Merge Engine	Merge Profiles
Validator	Validate Schema
Projector	Runtime Projection

1. Recruiter Parser

The Recruiter Parser reads recruiter data from CSV files.

Extracted fields include:

- Full Name
- Email
- Phone Number
- Current Company
- Job Title

Output:

```
Recruiter CSV
        │
        ▼
Structured Recruiter Object
```

---

2. Resume Parser

The Resume Parser extracts information from PDF resumes.

Extracted information:

- Candidate Name
- Email Addresses
- Phone Numbers
- LinkedIn URL
- GitHub URL
- Skills
- Experience
- Education

Output:

```
Resume PDF
      |
      ▼
Structured Resume Object
```

---

3. Normalization Layer

Different data sources often store values in different formats.

Normalization converts them into a common representation.

Phone Normalization

Example

```
+91 8986145487

↓

+918986145487
```

Date Normalization

Example

```
June 2025

↓

2025-06
```

Skill Normalization

Example

```
React.js
```

```
↓
React
```

---

4. Merge Engine

The Merge Engine combines recruiter and resume information into a unified canonical profile.

Responsibilities:

- Remove duplicates
- Resolve conflicting values
- Merge arrays
- Track provenance
- Calculate confidence

---

5. Provenance Engine

Every extracted field records its origin.

Each provenance record stores:

- Field
- Value
- Source
- Extraction Method
- Confidence
- Timestamp

Example

```
{
    "field": "skills[0]",
    "value": "Java",
    "source": "Resume",
    "method": "Dictionary Matching",
    "confidence": 0.85
}
```

---

6. Confidence Engine

Every source contributes a confidence score.

Current configuration:

Source	Confidence
Recruiter CSV	0.95
Resume	0.85

Overall confidence is calculated using the average confidence of all provenance entries.

---

7. Schema Validation

The merged canonical profile is validated using AJV.

Validation checks:

- Required fields
- Correct data types
- Valid object structure

Invalid profiles are rejected before projection.

---

8. Runtime Projection

The Projection Layer creates client-specific outputs using `config.json`.

Example configuration

```
{
    "path": "email",
    "from": "emails[0]"
}
```

Different clients can request different output formats without changing application logic.

Step 3 – Data Flow

The complete processing flow is:

```
Recruiter CSV
        │
        ▼
Recruiter Parser
        │
Resume PDF
        ▼
Resume Parser
        │
        ▼
Normalization
        │
        ▼
Merge Engine
```

```
        │
        ▼
Canonical Profile
        │
        ▼
Confidence Assignment
        │
        ▼
Provenance Generation
        │
        ▼
Schema Validation
        │
        ▼
Runtime Projection
        │
        ▼
Output JSON
```

Merge Strategy

The merge engine follows these rules.

Full Name

Preference:

```
Resume
↓

Recruiter CSV
```

Emails

- Merge all unique emails
- Remove duplicates

Phone Numbers

- Normalize to E.164 format
- Remove duplicates

Skills

- Normalize skill names
- Remove duplicates
- Sort alphabetically

Experience

Resume experience is added to the canonical profile.

Education

Resume education is added to the canonical profile.

Links

Merge:

- LinkedIn
- GitHub
- Portfolio

Validation Strategy

Validation ensures:

- Required fields exist
- Arrays contain valid values
- Nested objects follow schema
- Profile is structurally consistent

Assumptions

The implementation assumes:

- Resume contains selectable text.
- Recruiter CSV follows the expected schema.
- Dates follow Month-Year format.
- Skills belong to the predefined dictionary.
- Phone numbers can be normalized.
- Missing values are handled by runtime projection.

Limitations

Current implementation limitations:

- Regex-based parsing depends on resume formatting.
- OCR is not supported.
- LinkedIn and GitHub APIs are not integrated.
- Skills outside the dictionary are ignored.
- Confidence uses fixed source weights.

Future Improvements

Possible enhancements:

- OCR support for scanned resumes
- LinkedIn API integration
- GitHub API integration
- AI-powered resume parsing
- Machine-learning confidence scoring
- Location extraction
- Company extraction
- Years of experience calculation
- Multi-language resume parsing
- REST API deployment
- Web dashboard

Technologies Used

- Node.js
- JavaScript (ES6)
- pdf-parse
- csv-parser
- AJV
- File System (fs)

Folder Structure

```
eightfold-profile-engine/

│
├── input/
│   ├── recruiter.csv
│   └── resume.pdf
│
├── output/
│   ├── canonical-profile.json
│   └── projected-profile.json
│
├── src/
│   ├── merge/
│   ├── normalizers/
│   ├── parsers/
│   ├── projection/
│   ├── schema/
│   └── validators/
│
├── tests/
│   ├── confidence.test.js
│   ├── dateNormalizer.test.js
│   ├── mergeProfiles.test.js
│   ├── phoneNormalizer.test.js
│   ├── projector.test.js
│   ├── schemaValidator.test.js
│   └── skillNormalizer.test.js
│
├── .gitignore
├── config.json
├── package.json
├── DESIGN_DOCUMENT.md
└── README.md
```

Conclusion

The Eightfold Profile Engine implements a complete profile aggregation pipeline capable of extracting candidate information from structured and unstructured sources, normalizing data into a consistent format, merging records into a canonical profile, tracking provenance, calculating confidence scores, validating the merged profile, and generating configurable client-specific outputs through a runtime projection layer.

The modular architecture allows additional parsers, normalization strategies, and external profile sources to be integrated with minimal changes, making the system scalable and extensible for future enhancements.